SageV-Batch: Data-Parallel Cycle-Accurate Simulation of a Pipelined RV32I Processor on an AMD GPU

Lance Xavier Lim, James Russel Padua

Abstract

SageV is a web-based, cycle-accurate simulator of a five-stage pipelined RV32I processor and the foundation of an intelligent tutoring system for computer architecture. Its core runs one program at a time, which is enough for a student stepping through a pipeline. The planned tutoring features need far more. Randomized testing checks the simulator against a reference model over large sets of generated programs, and every generated practice exercise must be run to confirm that it produces the hazard it is meant to teach. SageV-Batch is a data-parallel engine that simulates thousands of RV32I programs at once on an AMD GPU through HIP, with one GPU thread for each simulated machine. We will rewrite the cycle step in a branch-free, predicated form over a structure-of-arrays layout, so that threads running together can execute different instructions without slowing each other down. Results will be checked against SageV's existing TypeScript core, and execution time will be compared with a sequential C++ version across increasing batch sizes.

Description of the Proposed Project

SageV-Batch runs SageV's cycle-accurate pipeline model on thousands of independent RV32I programs at the same time. SageV is a browser-based simulator written in TypeScript. It models a subset of RV32I [2], fifteen instructions plus the NOP pseudo-instruction, on a five-stage pipeline based on Patterson and Hennessy [1]. Results are forwarded into EX, branches are resolved in ID under predict-not-taken, and a load-use hazard costs one bubble. Each simulated machine has 256 bytes of memory, split into a 128-byte data segment and a 128-byte text segment that holds up to 32 instructions.

The simulator steps one program, one cycle per call. Two features planned for the tutoring system need many programs instead. The first is randomized verification, where the pipeline is compared against a sequential reference model over large sets of generated programs. One run during development checked about 51,000 different programs and 670,000 simulated cycles, all executed one after another on the CPU. The second is exercise generation. Every generated program has to be run to confirm that it produces the hazard it is meant to teach. In both cases, the work consists of many short simulations that do not depend on each other, where data-level parallelism applies.

Inputs

The engine takes a batch of N programs. Each program comes as a text-segment image produced by SageV's existing assembler, along with its starting data segment and register file. A batch file packs these images together with a cycle limit for each run, so a program that loops forever still stops. Programs come from random generation for verification and from exercises generated for the tutor.

Proposed Process

Processes to be parallelized with SIMT

The cycle step of the pipeline model is the part that runs in parallel. Each GPU thread handles one simulated machine and advances it one cycle at a time until it halts or reaches the cycle limit. In every

cycle, a thread does what SageV's core does: write-back, memory access, execution with forwarding, decode with hazard detection and branch resolution, and fetch. After all machines finish, each thread compares its final state with the expected state for its program, so the pass/fail check also runs on the GPU.

Related works

Instruction set simulation on GPUs has been done before. Raghav et al. [3] built an instruction set simulator on GPGPUs for thousand-core architectures, where the GPU simulates the many cores of one large target system. SageV-Batch does the reverse. It simulates many small single-core machines that are unrelated to each other, each with its own program.

Batch-stimulus simulation is closer to this workload. RTLflow [4] converts a register-transfer-level design into CUDA kernels and simulates it under many input stimuli at once. On the NVDLA design with 65,536 stimuli, it reports a 40x speedup over an 80-thread CPU baseline. Chen et al. [5] apply a similar idea to fuzzing: they translate smart-contract bytecode into a GPU program so that thousands of GPU cores test one contract at the same time, reaching 8.69x to 15.38x the throughput of earlier fuzzers. In both works, every thread runs the same design or the same contract on different inputs. Here, every thread runs a different program, so threads in the same thread group often decode different instructions in the same cycle. This divergence is the main difficulty of the project, and the Parallelization plan section describes how the step function is designed around it. Neither work targets an educational pipeline model or an AMD GPU.

Parallelization plan

Each simulated machine is mapped to one GPU thread. The step function is rewritten so that threads running different instructions still follow one instruction stream. Figure 1 shows the path of a batch from assembly to the final timing comparison.

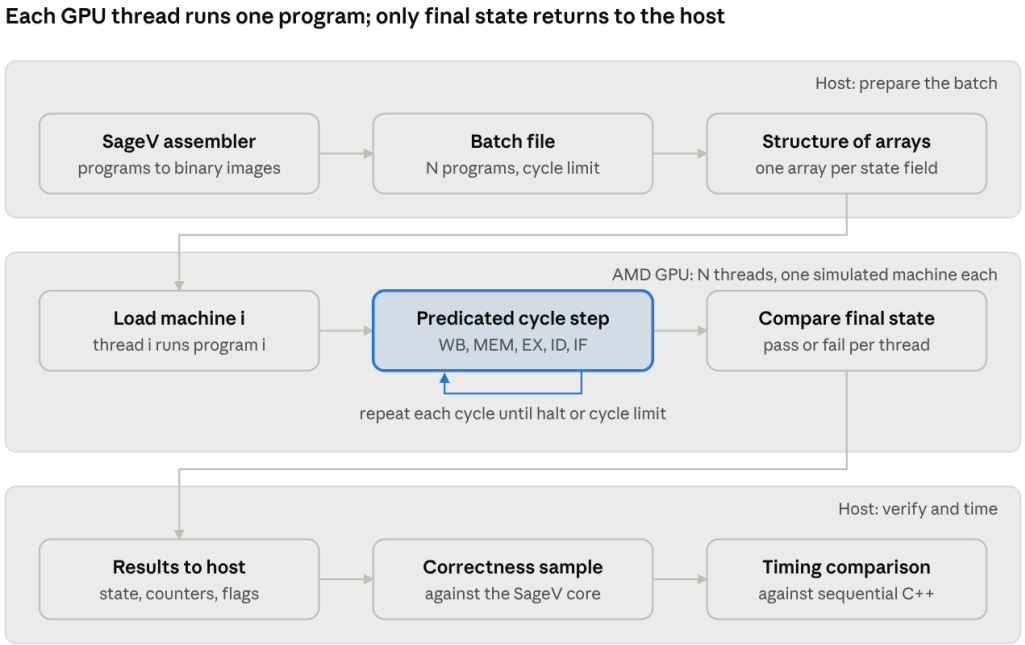


Figure 1. SageV-Batch data flow, host to GPU and back
Only the program images are sent to the GPU, and only final states, counters, and flags are sent back.

Data layout. Machine state is stored as a structure of arrays. Each architectural or pipeline-register field is one array of N entries, indexed by thread, so neighboring threads that read the same field touch neighboring words and their accesses come together. Reads that depend on a program's own bits, such as the register named by rs1 or a load address, become scattered reads from different addresses. The data segments are stored the same way, with each entry placed next to the neighboring thread's. We will measure whether they perform better in the local data share (LDS) or in global memory.

Predicated step. SageV's TypeScript core uses branches to choose an instruction type, and again to decide when to forward or stall. On a GPU, these branches split a thread group whenever its threads hold different instructions, and here that is the normal case. The kernel instead decodes each instruction into true/false masks, computes the small set of ALU operations, and picks the result with the masks. Forwarding becomes a pair of selects on the operand inputs. A stall becomes a flag that turns off the write enables of the PC and IF/ID. Every thread then runs the same instructions in every cycle, and only the selected values differ.

Termination. Programs halt at different cycles, and a thread group stays on the GPU until its last thread halts or reaches the cycle limit. Threads that finish early are switched off and do no work. We will measure the cost of these uneven finish times and test whether grouping programs of similar length into the same thread group reduces it.

Measurement. Kernel time will be recorded with HIP events, and end-to-end time with the host clock. Batch sizes start at 1,024 programs and double until GPU memory runs out, and each batch is also timed on the sequential C++ baseline. A random sample from every batch is run through SageV's TypeScript core as well, and its final states and counters must match exactly.

Processes that will not be parallelized

Program preparation stays on the host. SageV's TypeScript tools generate and assemble random programs sequentially, and the batch file is written once. This step costs little compared to the simulation and is not part of the measured time.

SageV's interactive layer is also left as it is. The web interface keeps a full history of every cycle so it can draw the pipeline map and label each pipeline-register field. The batch engine drops that history and keeps only each machine's final state and counters. A per-cycle record for every thread would greatly increase memory use, and neither workload needs one. The host analyses the results, including the timing comparison, after the kernel returns.

Outputs

For each program in the batch, the engine reports its final architectural state and the same counts that SageV's interface shows.

Output	Contents	Used for
Final architectural state	registers x0 to x31 and the 128-byte data segment	comparison against the sequential reference

Output	Contents	Used for
Pipeline counters	cycles, instructions retired, stall cycles, flushes	confirming that an exercise produces its intended hazard
Termination status	halted, or stopped at the cycle limit	separating looping programs from completed ones
Pass/fail flag	final state against the expected state	the verification report

For the whole batch, the engine writes a mismatch report that lists the programs that failed, and a timing table that compares the sequential C++ version with the HIP version across batch sizes. Kernel time and end-to-end time, which includes copies between the CPU and GPU, are reported separately.

Proposed Target Implementation Platform

The engine runs on an AMD GPU through HIP, AMD's C++ runtime and kernel language for GPU programming, which comes with ROCm. HIP uses the same SIMT model as CUDA. Kernels are written as code for one thread and run on many threads, and the hardware issues each instruction to a group of threads at the same time [6]. This thread group has 32 threads on RDNA GPUs and 64 on CDNA GPUs. If threads in a thread group take different branches, the hardware still processes the whole group and throws away the results of the threads that are switched off [6]. The predicated step function in the Parallelization plan section is meant to reduce that cost.

Component	Specification
GPU	AMD Radeon RX 6600 XT, 8 GB (RDNA 2, LLVM target gfx1032, 32-thread groups)
CPU (host and sequential baseline)	AMD Ryzen 5 5600G with Radeon Graphics, 3.90 GHz
System memory	16 GB
Operating system	Windows 11
GPU programming model	HIP (SIMT), HIP SDK for Windows 6.2.4
Host language	C++
Sequential baseline	single-threaded C++ implementation of the same step function
Correctness reference	SageV's TypeScript core, on a sample of each batch
Program preparation	SageV's existing TypeScript assembler and random program generator

The sequential baseline and the GPU kernel follow the same specification of the step function. Any difference in timing therefore comes from parallel execution, not from a different algorithm.

AMD supports this GPU only partly on Windows, so the HIP version matters. In the HIP SDK 6.2.4 requirements, the RX 6600 XT (gfx1032) is supported by the HIP runtime but not by the HIP SDK libraries [7]. The 7.x releases list it as unsupported [8]. SageV-Batch needs the runtime and the compiler but not the SDK's math libraries, so the project will use HIP SDK 6.2.4. A one-kernel test on the target machine during the first week will confirm this before implementation starts. The Ryzen 5 5600G also has an integrated GPU, so the host code will select the RX 6600 XT directly instead of using the default device.

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 2nd ed. Morgan Kaufmann (Elsevier), 2020. Publisher page

[2] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213. RISC-V Foundation, Dec. 2019. PDF

[3] S. Raghav, M. Ruggiero, D. Atienza, C. Pinto, A. Marongiu, and L. Benini, "Scalable instruction set simulator for thousand-core architectures running on GPGPUs," in *Proc. Int. Conf. High Performance Computing and Simulation (HPCS)*, 2010, pp. 459–466. Record

[4] D.-L. Lin, M. H. Ren, Y. Zhang, B. Khailany, and T.-W. Huang, "From RTL to CUDA: A GPU acceleration flow for RTL simulation with batch stimulus," in *Proc. 51st Int. Conf. Parallel Processing (ICPP)*, 2022, doi: 10.1145/3545008.3545091. Publication page

[5] W. Chen, X. Luo, H. Cai, and H. Wang, "Towards smart contract fuzzing on GPUs," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, doi: 10.1109/SP54263.2024.00229. Record

[6] Advanced Micro Devices, "HIP programming model," *ROCm Documentation*. Accessed: Oct. 13, 2026. Online

[7] Advanced Micro Devices, "System requirements (Windows)," *HIP SDK for Windows 6.2.4 Documentation*. Accessed: Oct. 13, 2026. Online

[8] Advanced Micro Devices, "System requirements (Windows)," *HIP SDK for Windows 7.1.1 Documentation*. Accessed: Oct. 13, 2026. Online